

REQUIREMENT ANALYSIS DOCUMENT

Movie Ticket Booking System

Course: B. Tech CSE

Semester: 5th

Subject: System Design

Subject Code: TCS-504

Assignment: Assignment 1

Introduction

The proposed system is a **Movie Ticket Booking System** for a single cinema. The system will be implemented as a **menu-driven C++ console application**.

The purpose of the system is to allow a customer to view currently playing movies, select a show, check seat availability, book one or more seats, make a payment, receive a ticket, and cancel a booking when required. The system will support three payment methods: **UPI, Card, and Cash**.

The system is intentionally limited to the functionality specified in the assignment. No additional features outside the defined scope will be implemented.

Scope of the System

The system will be designed for a **single cinema** and will provide the following eight core features:

1. List all movies currently playing.
2. Display shows available for a selected movie, including screen and start time.
3. Display the seat layout of a selected show with AVAILABLE/BOOKED status.
4. Book one or more seats while rejecting already-booked seats.
5. Calculate the booking price based on seat type.
6. Accept payment through UPI, Card, or Cash.
7. Print a ticket after successful payment.
8. Cancel a booking and make its seats available again.

Seat Type	Price
SILVER	₹150
GOLD	₹250
PLATINUM	₹400

Functional Requirements

Functional requirements describe **what the system shall do**. Each requirement is written so that it can be tested during the demonstration.

FR1 — Movie Listing

The system shall display all movies currently playing in the cinema, including each movie's **title, language, and duration**.

Expected Behaviour:

When the customer selects the movie-listing option, the system shall display all currently playing movies along with their title, language, and duration.

Test Condition:

The customer selects the Movies option and the system displays every currently playing movie with its title, language, and duration.

FR2 — Show Listing

For a selected movie, the system shall display all available shows along with their **screen number and start time**.

Expected Behaviour:

When the customer selects a movie, the system shall display the shows available for that movie.

Each show shall contain:

- Screen number
- Start time

Test Condition:

The customer selects a movie and the system displays all shows associated with that movie along with their screen and start time.

FR3 — Seat Availability

For a selected show, the system shall display the complete seat layout and show the status of each seat as either **AVAILABLE** or **BOOKED**.

Expected Behaviour:

When the customer selects a show, the system shall display all seats belonging to that show and indicate whether each seat is currently available or booked.

Test Condition:

The customer selects a show and the system displays every seat with its current availability status.

FR4 — Booking

A customer shall be able to select one or more seat numbers for a show. If **any selected seat is already BOOKED**, the whole booking shall be rejected and **no seat shall change its state**. A booking shall be confirmed only after payment succeeds.

Expected Behaviour:

- The customer selects one or more seats.

- The system checks the availability of every selected seat.
- If all selected seats are available, the booking process proceeds to payment.
- If even one selected seat is already booked, the complete booking is rejected.
- No selected seat shall be changed when the booking is rejected.
- The booking shall be confirmed only after successful payment.

Test Condition:

If the customer selects an already-booked seat along with any other seat, the system shall reject the entire booking and leave all seats unchanged.

FR5 — Price Calculation

The system shall calculate the total booking amount according to the type of each selected seat.

The prices shall be:

- **SILVER = ₹150**
- **GOLD = ₹250**
- **PLATINUM = ₹400**

Expected Behaviour:

The system shall determine the price of each selected seat according to its seat type and calculate the total amount of the booking.

Test Condition:

For every selected seat, the correct seat-type price shall be applied and the sum of all selected seats shall be displayed as the total booking amount.

FR6 — Payment

Exactly one payment method shall be selected for each booking from **UPI, Card, or Cash**. If the payment fails, the selected seats shall be released and the booking status shall become **FAILED**.

Expected Behaviour:

1. The system shall allow the customer to select exactly one payment method.
2. The supported methods shall be:
 - UPI
 - Card
 - Cash
3. The system shall attempt to process the payment.

4. If payment succeeds, the booking shall be confirmed.
5. If payment fails:
 - The booking shall not be confirmed.
 - The selected seats shall be released.
 - The booking status shall become FAILED.

Test Condition:

A simulated failed payment shall result in the seats becoming AVAILABLE again and the booking remaining unconfirmed

FR7 — Ticket Generation

After successful payment, the system shall print a ticket containing the following information:

- Booking ID
- Movie name
- Screen number
- Show time
- Selected seat numbers
- Total amount

Expected Behaviour:

The system shall generate and print the ticket only after successful payment and booking confirmation.

Test

Condition:

After successful payment, the displayed ticket shall contain the booking ID, movie, screen, show time, seats, and total amount. The expected demonstration output in the assignment follows this structure.

FR8 — Booking Cancellation

The system shall allow a customer to cancel a confirmed booking. When a booking is cancelled, all seats associated with that booking shall become AVAILABLE again.

Expected Behaviour:

1. The customer selects a booking for cancellation.
2. The system identifies the seats associated with the booking.
3. The booking is cancelled.
4. All seats belonging to that booking are made AVAILABLE again.

Test Condition:

After cancelling a booking, the customer can display the corresponding show's seat layout and verify that the previously booked seats are AVAILABLE.

Non-Functional Requirements

Non-functional requirements describe the **quality attributes and design constraints** of the system. The assignment specifically highlights modularity, extensibility, and validation of invalid input.

NFR1 — Modularity

The system shall be divided into independent modules, with **one class implemented per source file**. Each class shall have one clearly defined responsibility.

The system shall avoid placing all functionality inside a single large source file.

Verification:

Each major class such as Movie, Seat, Screen, Show, Booking, and BookingService shall be implemented separately according to the required modular structure.

The course specifically requires **one class per file and no header files**.

NFR2 — Extensibility

The payment system shall be designed so that a new payment method can be added without modifying the existing payment implementations or the core booking workflow.

For example, if **NetBanking** needs to be added in the future, a new payment class can be created that follows the existing Payment abstraction.

Verification:

Adding a NetBankingPayment class should not require changes to the existing BookingService payment-processing logic.

This requirement follows the assignment's Open/Closed Principle guidance.

NFR3 — Input Validation

The system shall validate user input such as menu choices, movie selections, show selections, and seat numbers and shall display a clear error message when invalid input is entered.

Invalid input shall not cause the program to terminate unexpectedly.

Verification:

The system shall handle:

- Invalid menu choices

- Invalid movie numbers
- Invalid show numbers
- Invalid seat numbers

and display an appropriate error message before allowing the user to continue.

The assignment explicitly requires invalid seat numbers or menu choices to be handled with a clear message and without a crash

NFR4 — Maintainability and Clean Code

The system shall use meaningful names, single-purpose functions, named constants, and minimal code duplication to make the code easy to understand and maintain.

The implementation shall:

- Use intention-revealing names.
- Avoid meaningless names such as temp1, list1, or c1.
- Keep each function focused on one responsibility.
- Avoid unnecessary levels of nested logic.
- Use constants instead of magic numbers.
- Avoid repeated code.
- Avoid unintended side effects between methods.

For example, seat prices shall be represented using named constants such as SILVER_PRICE, GOLD_PRICE, and PLATINUM_PRICE instead of directly repeating numeric values such as 150, 250, and 400.

Requirement Summary

Requirement ID	Requirement Name	Description
FR1	Movie Listing	Display currently playing movies with title, language and duration.
FR2	Show Listing	Display shows of a selected movie with screen and start time.
FR3	Seat Availability	Display seat layout with AVAILABLE/BOOKED status.
FR4	Booking	Select one or more seats and reject the entire booking if any selected seat is already booked.

Requirement ID	Requirement Name	Description
FR5	Pricing	Calculate price based on SILVER, GOLD and PLATINUM seat types.
FR6	Payment	Support UPI, Card and Cash; failed payment must release seats and fail the booking.
FR7	Ticket Generation	Print booking ID, movie, screen, time, seats and total amount.
FR8	Cancellation	Cancel a booking and make its seats AVAILABLE again.
NFR1	Modularity	One class per source file with clearly defined responsibilities.
NFR2	Extensibility	New payment methods can be added without modifying the booking workflow.
NFR3	Input Validation	Invalid input shall be handled clearly without crashing the program.
NFR4	Maintainability	Use clean, readable, modular and non-repetitive code.

Edge Cases to be Handled

The system shall specifically handle the following edge cases during demonstration:

Edge Case 1 — Already Booked Seat

If a customer attempts to book a seat that is already **BOOKED**, the booking shall be rejected and **no seat state shall change**.

Edge Case 2 — Failed Payment

If payment fails, the booking shall **not be confirmed** and all seats selected for that booking shall be released and become **AVAILABLE** again.

Edge Case 3 — Cancellation

If a confirmed booking is cancelled, all seats associated with that booking shall become **AVAILABLE** again.

Edge Case 4 — Invalid Input

If the customer enters an invalid menu choice or seat number, the system shall display a clear error message and continue execution without crashing.

Conclusion

The requirement analysis defines a focused movie ticket booking system for a single cinema. The system will provide movie and show discovery, seat availability checking, seat booking, price calculation, payment processing, ticket printing, and booking cancellation.

The functional requirements define the exact behaviour that will be implemented and tested. The non-functional requirements establish modularity, extensibility, input validation, and maintainability as important design constraints.

These requirements will serve as the foundation for the subsequent **Noun–Verb Analysis, class identification, relationship analysis, UML Class Diagram, Sequence Diagram, and modular C++ implementation.**